

PARTIU MARROCOS / KEROMINDPARTIU MARROCOS / KEROMIND

CRM Evolution 01 — Relatório de Fechamento

Relatório de Fechamento

15 de setembro de 2026

Etapa 3 de 5 — PM-NIGHT-RUN-01 (COMANDO DE EXECUÇÃO CONTINUADA CONTROLADA)

Executada sob autorização já concedida ("SE F2 = GREEN"), sem pausa para nova aprovação, conforme o comando vigente.

Objetivo

Evoluir o CRM de um Kanban simples para uma ferramenta comercial completa: detalhe de lead, timeline, filtros/busca, prioridade, motivo de perda, Note/Task tipados, follow-up melhorado, repescagem estruturada (Job Engine), lead scoring foundation e Next Best Action foundation — sem nunca dar ao agente Yalla poder de decisão/execução autônoma além do já estabelecido em T3.

Escopo autorizado (11 itens) × entregue

#	Item	Status
1	Página detalhada do lead (/leads/[id])	✓
2	Timeline/histórico	✓
3	Filtros	✓
4	Busca	✓
5	Prioridade	✓
6	Motivo de perda	✓
7	Task/Note tipados	✓
8	Follow-up melhorado	✓
9	Repescagem estruturada	✓
10	Lead scoring foundation	✓
11	Next Best Action foundation	✓

Todos os 11 itens do escopo foram entregues. Nenhum item foi cortado.

Arquitetura e decisões

Note/Task "tipo" — generalização de um prefixo textual

Antes: categorização existia só como prefixo de texto ([Yalla • Pendência]) dentro do campo **conteudo**. Agora: coluna real, tipada por enum — **Note.tipo** **NoteTipo?** (nulável — notas antigas ficam **null**, nunca reescritas) e **Task.tipo** **TaskTipo** **@default(GERAL)** (tarefas antigas viram **GERAL** por default, nunca reescritas). **NoteTipo** cobre **OBSERVACAO/INTERESSE/INTENCAO/PREFERENCIA/INFERENCIA/PENDENCIA/STAGE_CHANGE/HANDOFF/REPESCAMEM**; **TaskTipo** cobre **FOLLOWUP/GERAL**. Todos os pontos de escrita existentes (tools de T3: **nota.registrar**, **lead.mover_stage**, **lead.classificar**, **atendimento.encaminhar_humano**, **tarefa.criar**) foram atualizados para gravar o tipo correto.

Timeline — sem entidade Activity/Event nova

Decisão explícita conforme a orientação da autorização ("Audit técnico ≠ timeline comercial, não misturar sem justificativa"): a timeline do detalhe do lead é montada em memória, mesclando **Note + Task + Message** (das conversas do contato) já existentes, ordenados por data — nenhuma tabela nova, nenhuma duplicação de dado. O Audit Log (T1) continua sendo o registro técnico separado, nunca lido pela timeline comercial.

Repescagem estruturada — pipeline genérico via Job Engine (T5), não uma cópia do KeroSolar

Requisito explícito da autorização: "usar Job Engine T5, não copiar **reengage.ts** do KeroSolar, criar padrão genérico: regra → elegibilidade → job → Yalla compõe/prepara mensagem → policy → envio → audit". Implementado:

- **Regra/elegibilidade** (**diasDesdeUltimaAtividade**, **packages/db/src/crm/atividade.ts**): lead **ABERTO**, etapa não-terminal, ≥3 dias sem Note/Task/Message/criação, sem follow-up pendente.
- **Job** (**lead.repescar_elegibilidade**, **apps/web/src/lib/jobs/definitions/lead-repescar.ts**): roda no Job Engine real (T5) — **retry/backoff/timeout/lease** herdados de graça. **Reavalia a elegibilidade NA HORA de rodar**, nunca confia na condição de quando foi agendado (testado explicitamente: lead que avança para GANHO/PERDIDO ou recebe atividade nova entre o agendamento e a execução não é sinalizado).
- **Agendamento**: dois pontos de entrada equivalentes — a tool **lead.agendar_repescagem** (Yalla, via Tool Broker, capability própria, **requiresGate: false** porque o único efeito é "checar de novo depois") e a ação humana **agendarRepescagemLeadAction** (botão na página do lead, mesmo **submiterJob** direto, mesmo padrão de **cancelarJobAction/reenviarJobAction** já existentes em **actions/jobs.ts**). Ambos idempotentes por (**tenantId**, **leadId**, **dia**) — nunca duplicam o Job do mesmo dia.
- **Sinal**: se elegível, cria **Note{tipo: REPESCAGEM} + Task{tipo: FOLLOWUP}** — decisão humana ou do Yalla (via **tarefa.criar**/Tool Broker) decide o que fazer a partir daí.

Limitação honesta, registrada deliberadamente, não escondida: o último elo da cadeia descrita na autorização — "Yalla compõe/prepara mensagem → policy → envio" — **não foi implementado nesta rodada**. O job entrega a fundação real e testável (regra → elegibilidade → job → sinal), mas NÃO compõe nem envia mensagem autonomamente; isso ampliaria a autonomia do agente sobre uma ação de contato direto com o cliente sem que este ciclo tivesse testado esse caminho especificamente. Every cenário de teste do job (**job-lead-repescar.test.ts**) prova ativamente que **nenhuma `Message` é criada** em nenhum caso. Consistente com o mesmo princípio já aplicado ao Next Best Action ("ação recomendada ≠ execução automática").

Lead scoring — determinístico, sem IA, com motivo por fator

packages/db/src/crm/lead-scoring.ts — **calcularScoreLead**: soma pesos (completude de contato 15, completude de dados de viagem 30, progresso no funil 25, engajamento recente 20, origem rastreada 10 = 100 no total), cada fator com **motivo** explicável, nunca opaco. Não julga qual canal de origem é "melhor" — só premia ter atribuição rastreada ou não (mesmo princípio de neutralidade editorial já usado em T6). IA pode complementar no futuro; não implementado agora (fora de escopo).

Next Best Action — sugestão, nunca execução

packages/db/src/crm/next-best-action.ts — **sugerirProximaAcao**: retorna uma entre 6 ações

(**solicitar_informacao_faltante/agendar_retorno/encaminhar_humano/repescar_opportunidade/preparar_proposta/nenhuma_acao_necessaria**) + **motivo**. Puro (zero acesso a banco), nunca dispara nada sozinho — a página do lead só exibe a sugestão como painel informativo; quando a sugestão é "repescar oportunidade", um botão manual aciona o MESMO mecanismo de

agendamento descrito acima (nunca a sugestão executa por conta própria). Tool Broker/Gates continuam sendo o único caminho real de execução, sem mudança.

Página `/leads/[id]`

Consolida: contato, funil/etapa (com o mesmo seletor do Kanban), responsável, origem, valor, prioridade (toggle), motivo de perda (exibido quando presente), atribuição (T6), preferências de viagem declaradas, lead score (painel com breakdown por fator), Next Best Action (painel + botão condicional de repescagem), lista de tarefas (com checkbox de conclusão), e a timeline completa. Nenhuma tela nova reescreve dado já existente — tudo é leitura composta.

Busca/filtro em `/leads`

Filtro por nome/telefone do contato (case-insensitive), responsável (lista de usuários do tenant) e origem (valores distintos já usados pelo próprio tenant) — via `searchParams` da URL (server-side, sem estado de cliente escondido), aplicado antes do agrupamento por etapa do Kanban.

Motivo de perda na UI

O seletor de etapa (`MoverEtapaSelect`, reaproveitado tanto no Kanban quanto no detalhe do lead) agora pede o motivo via prompt nativo quando a etapa de destino é terminal de perda (`isLost`); cancelar o prompt cancela a troca de etapa. A regra de negócio (nunca gravar motivo em etapa não-terminal, nunca apagar um motivo já registrado por uma chamada sem motivo) é imposta no server action `moverLeadEtapa`, não confiando na UI — coberta por teste de integração direto na função (não só visualmente).

Migrations

1 nova, puramente aditiva, defaults seguros (`prioridade=false`, tarefas existentes viram `GERAL`, notas existentes ficam `tipo=null` — nenhum dado existente reescrito):

```
20260915020000_crm_evolution_01
CREATE TYPE "NoteTipo" ...
CREATE TYPE "TaskTipo" ...
ALTER TABLE "leads" ADD COLUMN "motivo_perda" TEXT, ADD COLUMN "prioridade" BOOLEAN NOT
NULL DEFAULT false
ALTER TABLE "notes" ADD COLUMN "tipo" "NoteTipo"
ALTER TABLE "tasks" ADD COLUMN "tipo" "TaskTipo" NOT NULL DEFAULT 'GERAL'
```

28 migrations no total (era 27 ao final da Etapa 2).

Segurança

- Toda leitura/escrita nova passa por `withTenant`/RLS — nenhuma query nova bypassa tenant.
- `lead.agendar_repescagem` é `SAFE_WRITE/requiresGate: false` (mesmo padrão de `task.criar`) — pior caso de uso indevido é reavaliar de novo mais tarde, sem efeito colateral externo algum.
- `agendarRepescagemLeadAction` (humano) exige `leads.manage`, mesmo padrão RBAC já usado por `cancelarJobAction/reenviarJobAction`.
- `motivoPerda` é sanitizado (remove HTML) e truncado (500 chars) antes de gravar — testado explicitamente contra payload `<script>`.
- Nenhuma tool/ação nova ganhou poder de enviar mensagem, aprovar a si mesma, ou pular Gate — Tool Broker (T3) e Gates (T1) inalterados nesta etapa.
- Nenhuma migration destrutiva; nenhum dado de produção tocado (schema local via `embedded-postgres`).

Testes novos (29)

- `packages/db/tests/unit/lead-scoring.test.ts` — 6
- `packages/db/tests/unit/next-best-action.test.ts` — 8
- `apps/web/tests/integration/job-lead-repesca.test.ts` — 8 (elegibilidade, reavaliação na hora de rodar, tool de agendamento, idempotência, default-deny, nunca envia mensagem)
- `apps/web/tests/integration/leads-actions.test.ts` — 7 (motivo de perda: grava/sanitiza/preserva; prioridade; conclusão de tarefa; RBAC)

Regressão

326/326 testes passando (297 ao final da Etapa 2 + 29 novos). Typecheck limpo (`packages/db` e `apps/web`). Build limpo — 17 rotas (`/leads/[id]` nova).

Verificação ponta a ponta (navegador real)

Servidor dev local iniciado, login com usuário demo, lead de teste criado com dados reais (contato, preferências, atribuição, notas, tarefa, mensagens de conversa). Confirmado visualmente:

- Detalhe do lead renderiza score (81/100 com breakdown por fator), Next Best Action ("Preparar proposta" — dados completos e follow-up já agendado, coerente com o esperado), atribuição, preferências, timeline mesclada (mensagens + tarefa + notas, ordem cronológica correta).
- Checkbox de tarefa alterna **concluída** de verdade (confirmado via estado do DOM pós-toggle).
- Busca por "fernanda" na listagem filtra corretamente para 1 resultado, badge de prioridade (●) visível no card.
- Nenhum erro no console do navegador em nenhuma das páginas testadas.

O fluxo de "motivo de perda via `window.prompt`" não foi exercido no navegador automatizado (risco real de travar a sessão de automação num diálogo nativo bloqueante) — em vez disso, a regra de negócio por trás dele (`moverLeadEtapa`) foi coberta por teste de integração direto (grava/sanitiza/preserva), que é a garantia que importa; a camada de UI é só um `window.prompt` simples, sem lógica própria.

Lead e usuário de teste (senha temporária) usados na verificação foram removidos/restaurados ao final — nenhum dado de demonstração permanente foi deixado no banco.

Limitações (honestas, não escondidas)

1. Repescagem nunca compõe/envia mensagem autonomamente (ver seção de arquitetura acima) — só sinaliza para decisão humana/Yalla via `tarefa.criar`.
2. Lead scoring é 100% determinístico — nenhum componente de IA/LLM contribui para o score nesta rodada (compatível com "IA pode complementar no futuro, não implementado agora" da autorização).
3. A timeline não pagina — carrega até 50 mensagens recentes por conversa; um lead com histórico muito longo pode não mostrar tudo (aceitável para foundation, paginação real fica para quando houver um caso de uso real que precise).
4. Busca é substring simples (contains, case-insensitive) — sem full-text search/ranking; suficiente para o volume atual.

Decisões do fundador pendentes

Nenhuma nova nesta etapa.

Resultado / Status

GREEN — implementação completa dentro do escopo (11/11 itens), testes novos passando (29/29), regressão completa passando (326/326), typecheck limpo, build limpo, migration consistente e não-

destrutiva, RLS protegido em toda tabela/query nova, nenhuma vulnerabilidade introduzida, nenhuma decisão do fundador indispensável, nenhum agente ganhou autoaprovação ou poder de envio autônomo, verificação ponta a ponta feita em navegador real.

Próximo bloco autorizado

Etapa 4 — Finance Core Foundation 02. **Prosseguindo automaticamente conforme autorização (PM-NIGHT-RUN-01).**